

Breast Cancer Classification Using Classical Machine Learning Models

Basic Applied Machine Learning Project

Dataset: Breast Cancer Wisconsin Diagnostic Dataset

Student: Fatemeh Bayat

May 29, 2026

Contents

1 Project Objective 2

2 Dataset Description 2

2.1 Dataset Size and Variables 2

2.2 Feature Groups 3

3 Understanding the Target Labels 3

4 Exploratory Data Analysis 3

4.1 Missing Values 3

4.2 Class Balance 3

4.3 Feature Ranges and Scaling Motivation 4

4.4 Feature Distribution Analysis 4

4.5 Correlation Analysis 6

5 Train-Test Split 7

6 Model Comparison with Cross-Validation 7

6.1 Compared Models 8

6.2 Cross-Validation Metrics 8

7 Hyperparameter Tuning 9

7.1 Logistic Regression Tuning 9

7.2 SVM Tuning 9

8 Final Model Selection 10

9 Final Evaluation on the Test Set 10

10 Confusion Matrix and Error Analysis 11

11 ROC Curve 11

12 Discussion 12

13 Limitations 13

14 Future Work 13

15 Conclusion 13

16 LLM Usage Declaration 13

1. Project Objective

The aim of this project was to classify breast cancer samples as **malignant** or **benign** using the Breast Cancer Wisconsin Diagnostic dataset. The task is a **supervised binary classification problem**, because each sample has numerical input features and a known diagnosis label.

The project follows a complete classical machine learning workflow:

- data loading and dataset inspection;
- exploratory data analysis (EDA);
- preprocessing and scale-aware modelling;
- stratified train-test split;
- model comparison using stratified cross-validation;
- hyperparameter tuning with `GridSearchCV`;
- final test-set evaluation;
- interpretation of errors, limitations, and possible future work.

Because this is a medical classification problem, performance was not evaluated using accuracy alone. The malignant class is clinically important because a malignant sample predicted as benign represents a possible missed cancer case. Therefore, malignant-specific recall, malignant F1-score, the confusion matrix, MCC, and ROC-AUC were also emphasized.

2. Dataset Description

The project used the **Breast Cancer Wisconsin Diagnostic dataset** available through `sklearn.datasets.load_wdbc`. The dataset contains measurements computed from digitized images of fine needle aspirate (FNA) biopsies of breast masses. Each sample is represented by numerical features describing properties of cell nuclei.

2.1. Dataset Size and Variables

The dataset contains:

- **569 samples**;
- **30 original numerical input features**;
- **1 target variable**;
- an additional human-readable diagnosis column added during the analysis.

Therefore, after adding the target and diagnosis columns, the working dataframe contained **32 columns**.

Table 1: Target label encoding used in the dataset.

Label	Diagnosis	Clinical meaning
0	Malignant	More clinically critical class
1	Benign	Non-malignant class

2.2. Feature Groups

The 30 numerical variables are based on cell nucleus properties such as radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension. These properties appear as mean, standard error, and worst-value measurements.

3. Understanding the Target Labels

A key methodological point in this project was the encoding of the target variable. In this dataset, **malignant is encoded as 0** and **benign is encoded as 1**. However, many scikit-learn metrics treat label 1 as the positive class by default.

For this reason, the project computed malignant-specific precision, recall, and F1-score using:

```
precision_score(y_test, y_pred_final, pos_label=0)
recall_score(y_test, y_pred_final, pos_label=0)
f1_score(y_test, y_pred_final, pos_label=0)
```

This was important because the malignant class is the clinically critical class. A false negative for malignant means that a malignant case was predicted as benign.

4. Exploratory Data Analysis

EDA was performed before model training to understand dataset quality, class distribution, feature ranges, and possible modelling challenges.

The EDA included:

- checking dataframe shape and data types;
- checking missing values;
- inspecting class balance;
- examining numerical feature ranges;
- visualizing selected feature distributions;
- calculating correlations between features.

4.1. Missing Values

The dataset was checked for missing values before modelling. No missing values were found, so no imputation strategy was needed. This simplified the preprocessing workflow and allowed all 569 samples to be kept.

4.2. Class Balance

The class distribution was moderately imbalanced, with more benign than malignant samples.

Table 2: Class distribution in the full dataset.

Class	Number of samples
Benign	357
Malignant	212

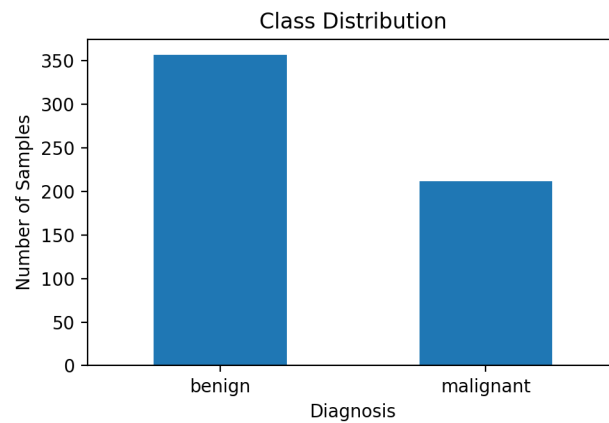


Figure 1: Class distribution of benign and malignant samples.

This imbalance shows why accuracy alone is not sufficient. A classifier could achieve good accuracy by performing well on the majority benign class while still missing malignant cases.

4.3. Feature Ranges and Scaling Motivation

The numerical features have very different value ranges. For example, area-related variables are much larger than smoothness or symmetry variables. This matters for algorithms that depend on distances, margins, or coefficient optimization.

The scale-sensitive models in this project were:

- Logistic Regression;
- K-Nearest Neighbors (KNN);
- Support Vector Machine (SVM).

For these models, scaling was applied inside `Pipeline` objects using `StandardScaler`. This prevented data leakage because the scaler was fitted only on the training data inside each cross-validation fold.

4.4. Feature Distribution Analysis

The project visualized selected biologically interpretable features to compare distributions between benign and malignant samples. The selected features were mean radius, mean texture, mean perimeter, mean area, and mean concave points.

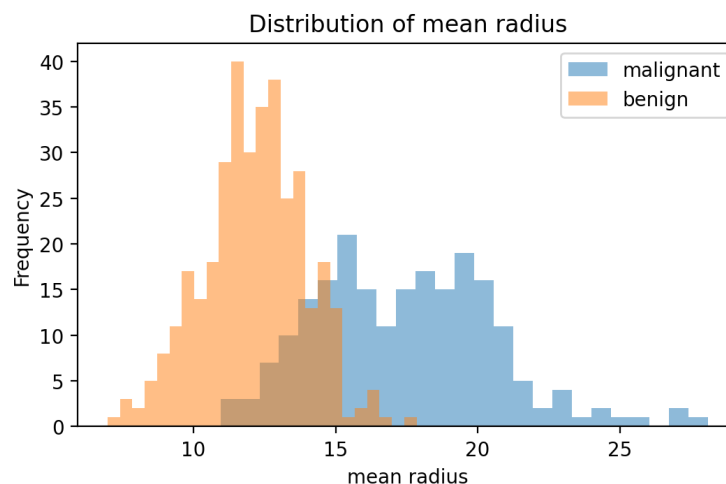


Figure 2: Distribution of mean radius by diagnosis.

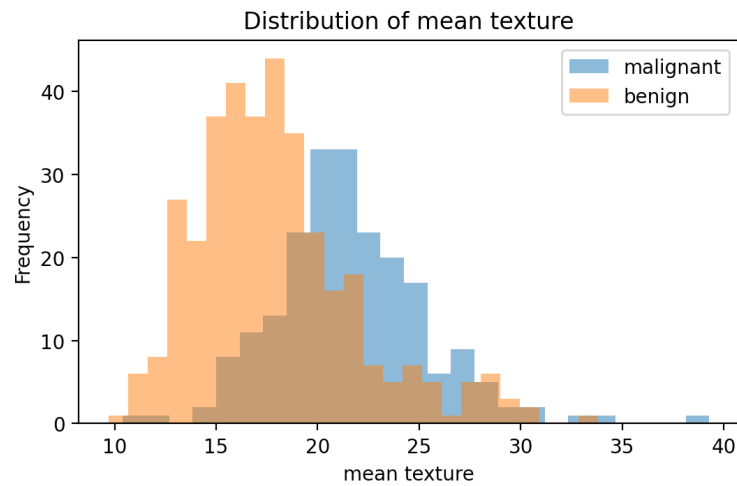


Figure 3: Distribution of mean texture by diagnosis.

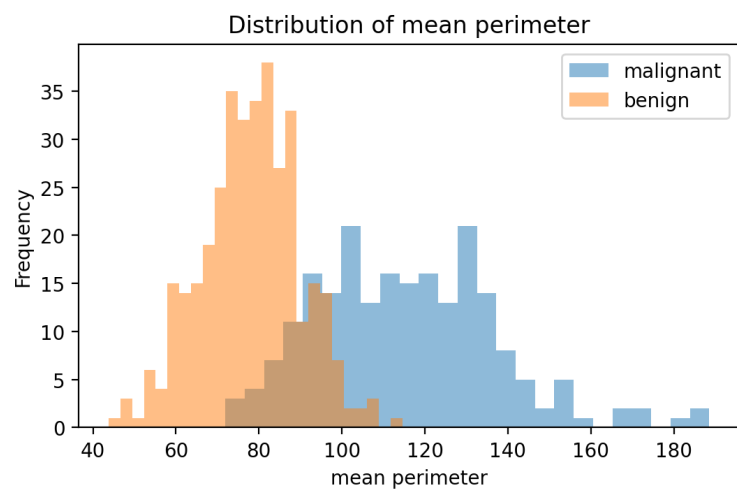


Figure 4: Distribution of mean perimeter by diagnosis.

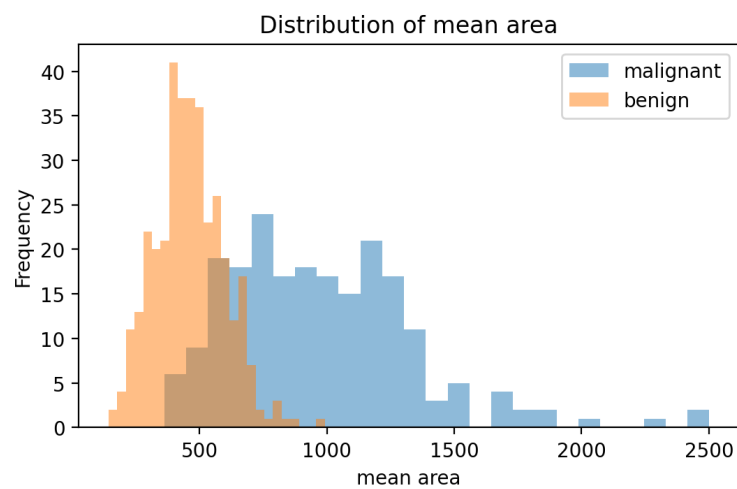


Figure 5: Distribution of mean area by diagnosis.

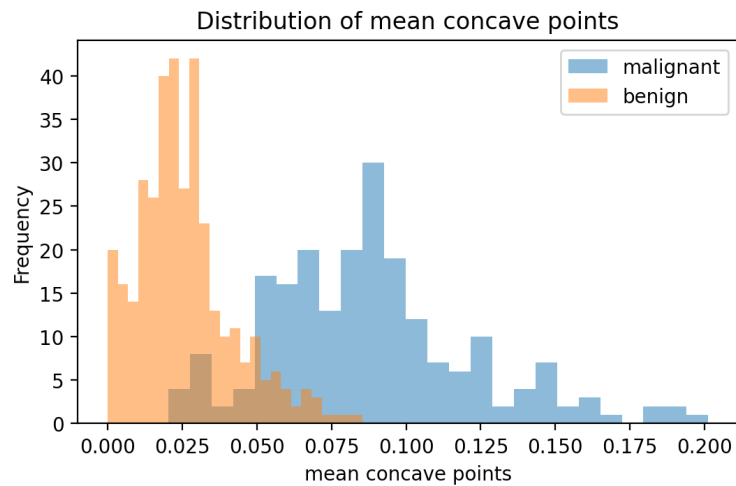


Figure 6: Distribution of mean concave points by diagnosis.

These plots show that several variables contain useful discriminatory information. However, there is still overlap between the two classes, so a multivariate machine learning model is needed.

4.5. Correlation Analysis

The dataset contains related measurements of similar biological properties. Therefore, high correlation between some variables was expected. For example, radius, perimeter, and area describe related size properties of cell nuclei.

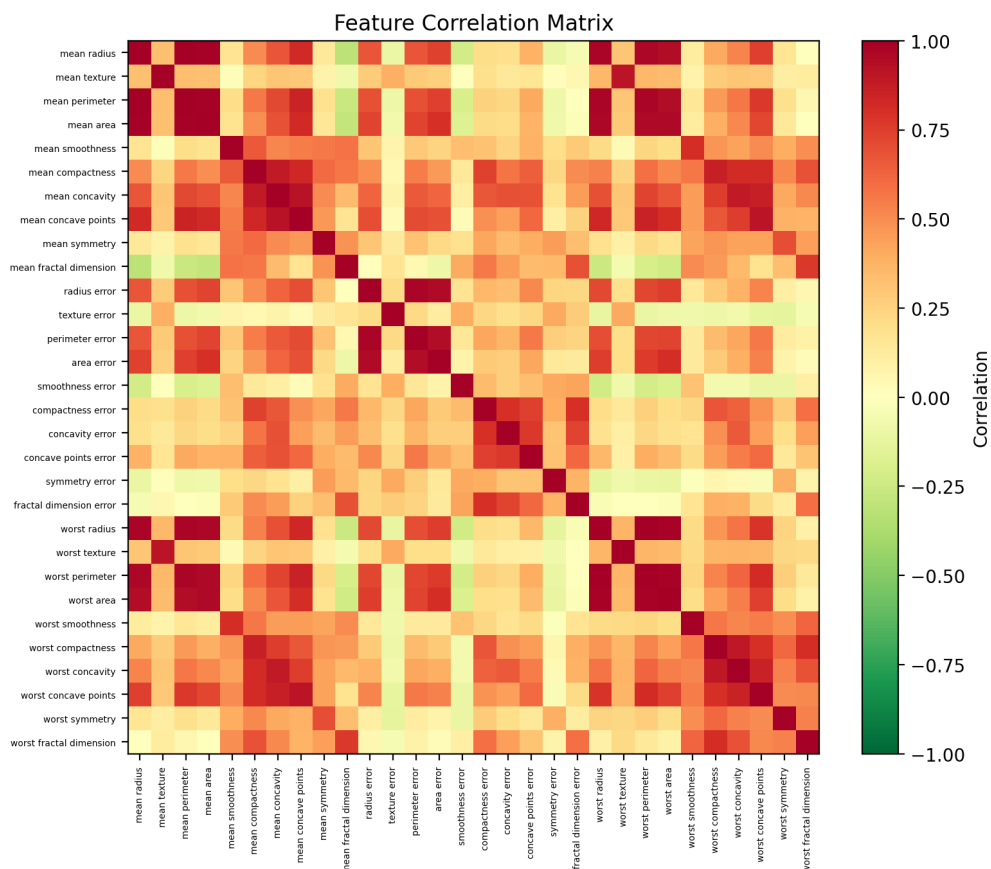


Figure 7: Correlation matrix of the 30 numerical input features.

Table 3: Examples of highly correlated feature pairs.

Feature 1	Feature 2	Correlation
Mean perimeter	Mean radius	0.9979
Worst perimeter	Worst radius	0.9937
Mean area	Mean radius	0.9874
Mean area	Mean perimeter	0.9865
Worst area	Worst radius	0.9840

Highly correlated features may provide overlapping information and can make interpretation more difficult. However, all features were initially kept because removing variables too early could remove useful predictive information. Regularization, cross-validation, and independent test evaluation were used to reduce overfitting risk.

5. Train-Test Split

Before model training, the dataset was split into training and test sets using a stratified split.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

Table 4: Train-test split configuration.

Setting	Value
Test size	0.20
Random state	42
Stratification	Yes, by target label
Training samples	455
Test samples	114

Table 5: Class distribution after stratified splitting.

Set	Benign	Malignant
Training set	285	170
Test set	72	42

The test set was kept unseen during model comparison and hyperparameter tuning. This was necessary to obtain a realistic final estimate of model performance.

6. Model Comparison with Cross-Validation

Several classical machine learning models were compared using **5-fold stratified cross-validation** on the training set only.


```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
malignant_recall_scorer = make_scorer(recall_score, pos_label=0)
```

6.1. Compared Models

Table 6: Models compared in the project.

Model	Reason for inclusion
Logistic Regression	Simple, interpretable, suitable for binary classification
KNN	Distance-based baseline model
SVM	Strong margin-based classifier, effective after scaling
Decision Tree	Interpretable non-linear model
Random Forest	Ensemble model, usually more stable than one tree

Scaling was included only where needed. Logistic Regression, KNN, and SVM were placed inside pipelines with `StandardScaler`. Decision Tree and Random Forest were used without scaling because tree-based models are not scale-sensitive in the same way.

6.2. Cross-Validation Metrics

The models were compared using:

- **F1-macro**, because it gives equal weight to both classes;
- **malignant recall**, because false negatives are clinically important.

Table 7: Model comparison using 5-fold stratified cross-validation.

Model	Mean F1-macro	SD F1-macro	Mean malignant recall	SD malignant recall
Logistic Regression	0.9764	0.0106	0.9647	0.0288
Random Forest	0.9603	0.0188	0.9588	0.0300
SVM	0.9669	0.0160	0.9529	0.0399
KNN	0.9598	0.0118	0.9294	0.0235
Decision Tree	0.9119	0.0190	0.9235	0.0353

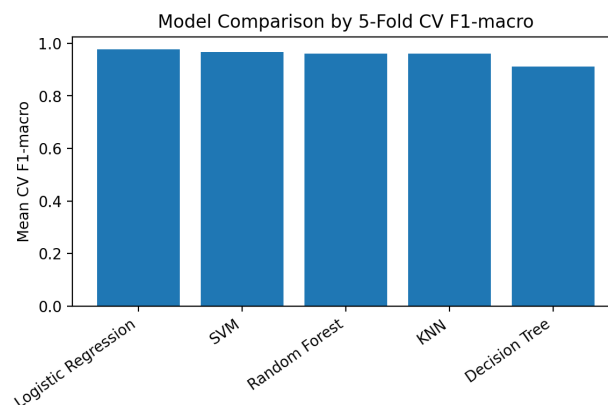


Figure 8: Model comparison by mean cross-validation F1-macro.

Logistic Regression was the strongest candidate model because it achieved the best mean F1-macro and the highest mean malignant recall in cross-validation.

7. Hyperparameter Tuning

After model comparison, hyperparameter tuning was applied to the two strongest candidate models: Logistic Regression and SVM. Tuning was performed using `GridSearchCV` with the same stratified cross-validation object.

7.1. Logistic Regression Tuning

The Logistic Regression pipeline used `StandardScaler` followed by `LogisticRegression(max_iter=1000, random_state=42)`.

```
logreg_param_grid = {
    'model__C': [0.01, 0.1, 1, 10, 100],
    'model__penalty': ['l2'],
    'model__solver': ['lbfgs']
}
```

Table 8: Best Logistic Regression hyperparameters.

Parameter	Best value
C	0.1
Penalty	l2
Solver	lbfgs
Best CV F1-macro	0.9811

The value of **C** controls inverse regularization strength. A smaller value means stronger regularization, which can help reduce overfitting.

7.2. SVM Tuning

The SVM pipeline used `StandardScaler` followed by `SVC(probability=True, random_state=42)`.

```
svm_param_grid = {
    'model__C': [0.1, 1, 10, 100],
    'model__kernel': ['linear', 'rbf'],
    'model__gamma': ['scale', 'auto']
}
```

Table 9: Best SVM hyperparameters.

Parameter	Best value
C	0.1
Kernel	linear
Gamma	scale
Best CV F1-macro	0.9739

Although SVM performed well, Logistic Regression had a higher tuned cross-validation F1-macro and better interpretability. Therefore, Logistic Regression was selected as the final model.

8. Final Model Selection

The final selected model was a Logistic Regression pipeline:

```
Pipeline([
  ('scaler', StandardScaler()),
  ('model', LogisticRegression(C=0.1,
                               penalty='l2',
                               solver='lbfgs',
                               max_iter=1000,
                               random_state=42))
])
```

This model was selected before evaluating the test set. The decision was based on:

- strongest cross-validation performance;
- high malignant recall;
- simple and interpretable model structure;
- suitability for a basic applied machine learning project.

9. Final Evaluation on the Test Set

The final Logistic Regression pipeline was evaluated once on the independent test set. The following metrics were calculated: accuracy, balanced accuracy, class-specific precision, class-specific recall, class-specific F1-score, MCC, ROC-AUC, and the confusion matrix.

Table 10: Final test-set performance of Logistic Regression.

Metric	Value
Accuracy	0.9737
Balanced accuracy	0.9692
Benign precision	0.9726
Benign recall	0.9861
Benign F1-score	0.9793
Malignant precision	0.9756
Malignant recall	0.9524
Malignant F1-score	0.9639
MCC	0.9433
ROC-AUC	0.9957

The final test results were strong. However, the malignant recall of 0.9524 means that the model did not detect every malignant case. This is important in a medical context.

10. Confusion Matrix and Error Analysis

Table 11: Final confusion matrix.

Actual / Predicted	Predicted malignant	Predicted benign
Actual malignant	40	2
Actual benign	1	71

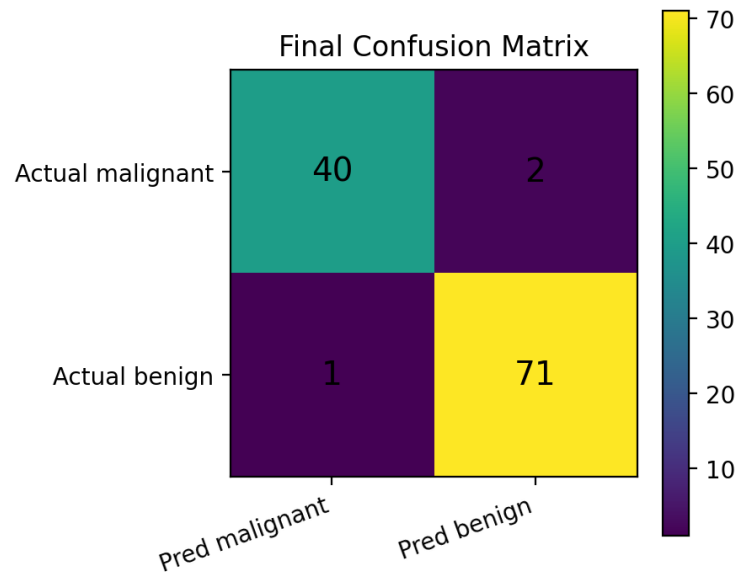


Figure 9: Final confusion matrix for the selected Logistic Regression model.

The model correctly classified 40 malignant samples and 71 benign samples. It misclassified 2 malignant samples as benign and 1 benign sample as malignant.

The most clinically important error type is the malignant false negative. In this test set, there were **2 false negatives**. This supports the decision to discuss malignant recall separately instead of relying only on accuracy.

11. ROC Curve

For the ROC curve, malignant was treated as the positive class. This required using the predicted probability for class 0 and converting the test labels so that malignant was represented as 1 for the ROC-AUC calculation.

```
y_proba_malignant = final_model.predict_proba(X_test)[: , 0]
y_test_malignant = (y_test == 0).astype(int)
final_roc_auc = roc_auc_score(y_test_malignant, y_proba_malignant)
```

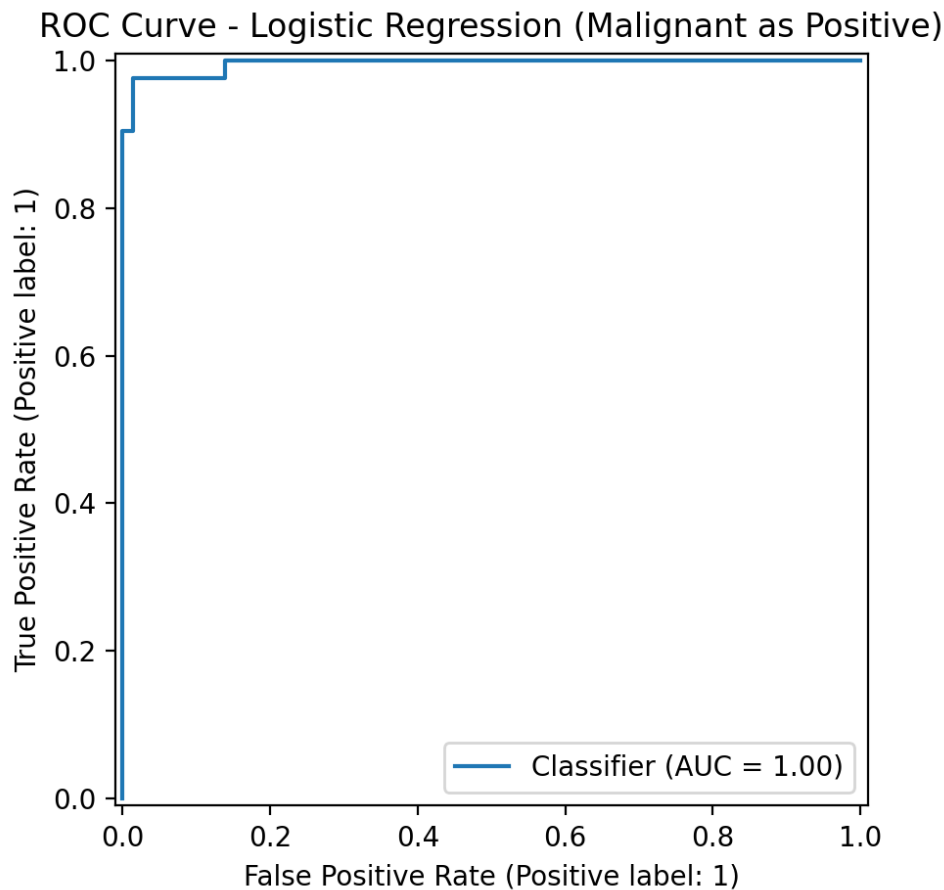


Figure 10: ROC curve for the final model, treating malignant as the positive class.

The ROC-AUC was 0.9957, indicating excellent separation between malignant and benign samples across classification thresholds. However, ROC-AUC should be interpreted together with the confusion matrix and malignant recall because error type matters in medical diagnosis.

12. Discussion

The results show that classical machine learning models can perform very well on this structured medical dataset. Logistic Regression performed best overall, combining strong predictive performance with interpretability.

The project used several methodologically important practices:

- stratified train-test splitting to preserve class proportions;
- stratified cross-validation for reliable model comparison;
- pipelines to prevent scaling-related data leakage;
- malignant-specific metrics because the default positive class in scikit-learn would otherwise be benign;
- independent final test-set evaluation after model selection.

Although the performance was strong, the model still produced 2 malignant false negatives. Therefore, this type of model should not be used alone in real clinical applications. It could be considered only as a decision-support tool together with expert medical assessment and external validation.

13. Limitations

This project has several limitations:

1. The dataset is relatively small, with only 569 samples.
2. The dataset is already clean and structured, while real clinical data may contain missing values, noise, batch effects, or measurement variability.
3. No external clinical validation dataset was used.
4. Highly correlated features were analyzed but not removed using a formal feature selection method.
5. Threshold optimization was discussed but not implemented as a final decision rule.
6. The project focused on classical machine learning models, which was appropriate for the course scope but not exhaustive.

14. Future Work

Possible future improvements include:

- testing the final model on an external clinical dataset;
- applying feature selection and comparing performance before and after feature reduction;
- performing threshold optimization to reduce malignant false negatives;
- comparing different scoring metrics during hyperparameter tuning;
- investigating probability calibration;
- analyzing Logistic Regression coefficients for interpretability.

Threshold optimization would be especially relevant because increasing malignant recall may reduce missed malignant cases, although this may increase false positives.

15. Conclusion

This project implemented a complete classical machine learning workflow for breast cancer classification. The Breast Cancer Wisconsin Diagnostic dataset contained 569 samples and 30 numerical features, with a moderately imbalanced target distribution.

After EDA, preprocessing, model comparison, cross-validation, hyperparameter tuning, and independent test-set evaluation, Logistic Regression was selected as the final model. The final test results were strong: accuracy was 0.9737, balanced accuracy was 0.9692, malignant recall was 0.9524, malignant F1-score was 0.9639, MCC was 0.9433, and ROC-AUC was 0.9957.

The final model made only 3 test-set errors, but 2 of them were malignant false negatives. Therefore, the model achieved high performance but still requires careful clinical interpretation and external validation.

16. LLM Usage Declaration

LLMs were used for code debugging, language polishing, refactoring, and improving the clarity of explanations. All methodological choices, experiments, outputs, and interpretations were reviewed, checked, and validated.